

**LAPORAN LAPORAN PROJECT UAS
MATA KULIAH PEMROGRAMAN BERBASIS WEB**



Disusun oleh

Nama : Muhammad Syafiq Zahran
Nomor Mahasiswa : 12241970
Program studi : Informatika
Jenjang : Sarjana

**SEKOLAH TINGGI MANAJEMEN INFORMATIKA DAN ILMU KOMPUTER
EL RAHMA YOGYAKARTA
YOGYAKARTA
2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perpustakaan sebagai pusat literasi dan sarana edukasi membutuhkan sistem pencatatan yang terstruktur untuk mengawasi keluar masuknya koleksi buku. Pencatatan manual yang mengandalkan buku besar (ledger) seringkali menimbulkan berbagai kendala, di antaranya adalah lamanya waktu pencarian data anggota, tingginya probabilitas kesalahan pencatatan transaksi (human error), dan sulitnya melakukan inventarisasi stok buku secara akurat (real-time). Kendala ini berdampak pada efisiensi staf administrasi serta layanan terhadap sirkulasi perpustakaan.

Oleh karena itu, diperlukan transformasi digital melalui pengembangan Sistem Informasi Perpustakaan berbasis Web. Sistem ini ditujukan sebagai perangkat lunak (software) penunjang utama yang akan memfasilitasi peran administrator dalam mengelola database secara terpadu. Framework Laravel digunakan sebagai tulang punggung arsitektur backend dikarenakan keandalannya dalam menangani operasi basis data tingkat tinggi dan kemampuannya untuk mengamankan data transaksi.

1.2 Maksud dan Tujuan Sistem

Pembangunan sistem ini dimaksudkan untuk mencapai efisiensi manajemen sirkulasi buku. Adapun tujuan spesifiknya meliputi:

1. Menyediakan pangkalan data terpusat (Centralized Database) untuk menyimpan katalog buku dan identitas anggota.
2. Menghindari manipulasi stok fisik dengan cara membuat sistem pengurang dan penambah stok yang terotomatisasi di dalam transaksi.
3. Memberikan pengalaman pengguna (User Experience) yang baik melalui antarmuka (User Interface) responsif yang menyesuaikan ukuran perangkat.
4. Mempersingkat waktu pembuatan rekapitulasi pelaporan melalui fitur cetak otomatis dokumen format PDF dan Excel.

1.3 Ruang Lingkup (Batasan Sistem)

Agar pengembangan sistem tetap fokus dan terarah, ditetapkan batasan ruang lingkup sebagai berikut:

- Sistem bersifat Single-User, yaitu aplikasi hanya dikelola oleh satu peran utama (Admin).
- Ruang lingkup pengolahan (CRUD) meliputi Data Buku, Data Anggota, dan Transaksi Peminjaman.
- Logika peminjaman membatasi bahwa status sirkulasi hanya mencakup status "Dipinjam" dan "Dikembalikan".

BAB II

ARSITEKTUR DAN STRUKTUR DATABASE

2.1 Alur Kerja Sistem (System Workflow)

Sistem ini dirancang dengan alur kerja yang saling berkesinambungan. Berikut adalah penjelasan alur bisnis dari masing-masing modul:

- **Alur Master Data:** Admin diharuskan mendaftarkan buku-buku baru ke dalam sistem sebelum dapat dipinjam. Pendaftaran mencakup input metadata (judul, penulis) beserta

jumlah ketersediaan fisik (stok). Begitu pula dengan keanggotaan, siswa/mahasiswa harus diregistrasikan ke dalam modul anggota terlebih dahulu agar memiliki ID Sistem.

- **Alur Peminjaman Buku:** Saat ada peminjaman, Admin membuka modul Peminjaman, memilih ID Anggota yang meminjam, lalu memilih ID Buku yang dipinjam beserta batas waktu pengembalian. Sistem akan melakukan validasi: Jika stok buku saat itu kurang dari atau sama dengan 0, transaksi akan digagalkan oleh sistem. Jika stok aman, sistem akan merekam transaksi, menetapkan status menjadi 'Dipinjam', lalu secara otomatis memotong stok fisik di tabel buku sejumlah 1.
- **Alur Pengembalian Buku:** Ketika buku dikembalikan, Admin hanya perlu mencari data peminjaman tersebut di dalam daftar dan menekan tombol 'Kembalikan'. Sistem akan mengubah status menjadi 'Dikembalikan' dan secara otomatis mendeteksi buku apa yang terkait dengan transaksi tersebut lalu menambahkan kembali stoknya (+1) di tabel Buku.
- **Alur Pelaporan:** Admin kapan saja dapat mengekspor data keseluruhan transaksi dengan satu kali klik yang akan memicu mesin server menghasilkan file berformat ``pdf`` atau ``xlsx``.

2.2 Struktur Tabel Basis Data (Relasi)

Basis data aplikasi menggunakan Relasi MySQL (RDBMS). Terdapat 3 tabel yang saling bersinggungan. Desain ini menggunakan metode kunci tamu (Foreign Key) untuk menghubungkan tabel transaksi (peminjamans) dengan dua tabel master (bukus dan anggotas).

1. Tabel bukus (Katalog Inventaris)

Tabel ini menyimpan keseluruhan informasi dari fisik buku. Pengaturan kolom sangat penting di sini, terutama kolom stok yang akan terus berubah secara dinamis seiring terjadinya sirkulasi pinjam-meminjam.

Nama Field	Type Data Constraint	Fungsi dan Keterangan Singkat
id	bigint (Primary Key, Auto Increment)	Sebagai kunci utama yang mewakili identitas spesifik sebuah buku di seluruh database.
kode_buku	varchar (Unique)	Nomor atau kode barcode buku. Ditetapkan unik agar tidak ada duplikasi buku ganda.
judul	varchar	Judul dari buku.
penulis	varchar	Pencipta / Penulis naskah buku.
penerbit	varchar	Instansi atau perusahaan yang mencetak buku.
tahun	integer	Angka tahun terbit buku (misal: 2023).

stok	integer	Indikator jumlah fisik buku di perpustakaan. Penting: Field ini selalu berubah sesuai transaksi.
cover	varchar (Nullable)	Menyimpan path alamat direktori server letak gambar cover buku disimpan.

2. Tabel anggota (Data Peminjam)

Tabel entitas yang merepresentasikan subjek (orang) yang melakukan peminjaman buku.

Nama Field	Tipe Data Constraint	Fungsi dan Keterangan Singkat
id	bigint (Primary Key, Auto Increment)	Kunci utama yang mewakili satu keanggotaan.
nama	varchar	Identitas nama lengkap anggota perpustakaan.
alamat	text	Alamat domisili lengkap anggota untuk keperluan korespondensi administrasi.
no_hp	varchar	Nomor telepon seluler.
email	varchar	Alamat email untuk kontak alternatif.

3. Tabel peminjamans (Tabel Transaksi Pivot)

Tabel ini adalah jantung dari aplikasi. Ini menghubungkan siapa (anggota) yang meminjam apa (buku), kapan, dan dalam status apa. Relasinya menggunakan kardinalitas *One-to-Many* (Satu anggota bisa meminjam banyak, satu buku bisa dipinjam dalam banyak transaksi sejarah).

Nama Field	Tipe Data Constraint	Fungsi dan Keterangan Singkat
id	bigint (Primary Key, Auto Increment)	Nomor unik per satu baris transaksi peminjaman.
anggota_id	bigint (Foreign Key)	Terhubung (References) ke kolom `id` pada tabel `anggotas`. Memuat siapa peminjamnya.
buku_id	bigint (Foreign Key)	Terhubung (References) ke kolom `id` pada tabel `bukus`. Memuat buku apa yang dipinjam.

tgl_pinjam	date	Tanggal terjadinya transaksi pengambilan fisik buku.
tgl_kembali	date	Target batas waktu maksimal buku wajib dikembalikan ke perpustakaan.
status	varchar	Kondisi buku (Nilai hanya boleh 'Dipinjam' atau 'Dikembalikan').

BAB III

IMPLEMENTASI KODE DAN PEMBEDAHAN ALUR LOGIKA

Pada bab ini, dokumentasi kode akan dijelaskan secara rinci. Kode-kode ini mencerminkan struktur **backend** dari Framework Laravel, dimulai dari pengaturan **Routing**, validasi form, pengunggahan media, hingga logika rumit pengendalian stok di **Controller** transaksi.

3.1 Infrastruktur Rute (Routing System)

Pengaturan URL rute diatur pada file `routes/web.php`. Aplikasi menggunakan pola `Route::resource()`. Pola ini sangat praktis dan efisien karena Laravel akan secara otomatis membaca 1 baris kode dan menerjemahkannya menjadi 7 rute standar API RESTful (Get Index, Get Create, Post Store, Get Show, Get Edit, Put Update, dan Delete Destroy) tanpa perlu menuliskannya satu persatu.

```
// File: routes/web.php use App\Http\Controllers\DashboardController; use
App\Http\Controllers\BukuController; use App\Http\Controllers\AnggotaController; use
App\Http\Controllers\PeminjamanController; // 1. Rute Dashboard (Entry Point)
Route::get('/', [DashboardController::class, 'index'])->name('dashboard'); // 2. Rute Master
Data & Transaksi (Menggunakan standar REST Resource) Route::resource('buku',
BukuController::class); Route::resource('anggota', AnggotaController::class);
Route::resource('peminjaman', PeminjamanController::class); // 3. Rute Tindakan Ekstra
(Custom Action) untuk Pengembalian Buku
Route::put('/peminjaman/{peminjaman}/kembali', [PeminjamanController::class,
'kembalikan'])->name('peminjaman.kembali'); // 4. Rute Ekspor Laporan
Route::get('/export/peminjaman/pdf', [PeminjamanController::class, 'exportPdf'])->
>name('peminjaman.export.pdf'); Route::get('/export/peminjaman/excel',
[PeminjamanController::class, 'exportExcel'])->name('peminjaman.export.excel');
```

Penjelasan: Rute no.3 secara eksplisit dibuat bertipe PUT karena aksi ini bersifat memodifikasi / mengupdate satu data spesifik di dalam tabel peminjaman (mengubah statusnya).

3.2 Modul Data Buku: Proses Upload Gambar dan Limitasi Pagination

Selain menyimpan teks (huruf dan angka), aplikasi diwajibkan mampu memproses **file digital** (gambar cover). Alur kodenya adalah: Sistem memverifikasi apakah tipe file sesuai (jpg, png) dan ukurannya tidak melebihi 2 Megabyte. Jika lolos, file akan disimpan secara fisik ke direktori `storage/app/public/covers`. Selain itu pada saat pemanggilan data di layar tabel, fungsi ``paginate(10)`` membatasi sistem agar hanya melempar maksimal 10 baris ke tabel layar untuk menjaga kecepatan pemrosesan (*Page Load*).

```
// File: app/Http/Controllers/BukuController.php // A. Metode Menampilkan Data (Termasuk Logika Kotak Pencarian / Search) public function index(Request $request) { $search = $request->get('search'); // Pencarian dengan operator 'LIKE' agar cocok meskipun huruf hanya terketik sebagian. $buku = Buku::when($search, function($query) use ($search) { return $query->where('judul', 'like', "%$search%") ->orWhere('penulis', 'like', "%$search%") ->orWhere('kode_buku', 'like', "%$search%"); }) ->latest() // Data diurutkan dari waktu pendaftaran terakhir (LIFO) ->paginate(10) // <-- FUNGSI PAGINATION (10 Data per Halaman UI) ->withQueryString(); return view('buku.index', compact('buku', 'search')); } // B. Metode Menyimpan Data Master Buku Baru public function store(Request $request) { // Validasi Keamanan (Security Constraints) $validated = $request->validate([ 'kode_buku' => 'required|unique:bukus', // Tidak boleh ada kode buku kembar (Unique) 'judul' => 'required|string', 'stok' => 'required|integer|min:0', // Stok tidak boleh angka negatif 'cover' => 'nullable|image|mimes:jpeg,png,jpg|max:2048' // Validasi Format Gambar ]); // Algoritma Pemrosesan Media Upload if ($request->hasFile('cover')) { // file() merubah objek file menjadi bitstream dan store() menyimpannya ke folder. $validated['cover'] = $request->file('cover')->store('covers', 'public'); } Buku::create($validated); return redirect()->route('buku.index')->with('success', 'Buku berhasil ditambahkan.');
```

3.3 Modul Transaksi Peminjaman: Mesin Manipulasi Stok (Core Logic)

Bagian ini adalah otak inti perpustakaan. Karena struktur database memisahkan peminjaman dan entitas buku, sistem harus mengkoneksikannya di level kodingan menggunakan fitur Eloquent ORM di Laravel.

```
// File: app/Models/Peminjaman.php // DEKLARASI RELASI DATABASE class Peminjaman extends Model { // 1. Relasi ke Tabel Anggota public function anggota() { // Sistem sekarang mengerti bahwa kolom 'anggota_id' adalah jembatan menuju Model Anggota return $this->belongsTo(Anggota::class, 'anggota_id'); } // 2. Relasi ke Tabel Buku public function buku() { // Sistem sekarang mengerti bahwa kolom 'buku_id' adalah jembatan menuju Model Buku return $this->belongsTo(Buku::class, 'buku_id');
```

Setelah dihubungkan, alur pencatatan peminjaman dan pengurangan stok diaplikasikan di PeminjamanController.php. Apabila stok kurang dari sama dengan 0 (habis), fungsi akan dicegat oleh blok *if statement* dan segera melempar pengguna (Admin) ke halaman sebelumnya disertai notifikasi kegagalan. Jika aman, stok dipotong menggunakan perintah spesifik `decrement('stok')`.

```
// File: app/Http/Controllers/PeminjamanController.php // LOGIKA SAAT TRANSAKSI BARU (BUKU DIPINJAM OLEH ANGGOTA) public function store(Request $request) { // Cek Form Input $request->validate([ 'anggota_id' => 'required|exists:anggotas,id', 'buku_id' => 'required|exists:bukus,id', 'tgl_pinjam' => 'required|date', 'tgl_kembali' => 'required|date|after_or_equal:tgl_pinjam', ]); // TAHAP 1: Proteksi Stok Ekstrem (Mencegah Peminjaman Buku Kosong) $buku = Buku::findOrFail($request->buku_id); if ($buku->stok <= 0) { // Mengembalikan proses (Abort) return redirect()->back()->with('error', 'Gagal meminjam: Stok buku secara fisik sudah habis!'); } // TAHAP 2: Injeksi Baris Data ke Database Peminjamans Peminjaman::create([ 'anggota_id' => $request->anggota_id, 'buku_id' => $request->buku_id, 'tgl_pinjam' => $request->tgl_pinjam, 'tgl_kembali' => $request->tgl_kembali, 'status' => 'Dipinjam', // Ditetapkan hard-code sebagai Dipinjam ]); // TAHAP 3: Auto-Decrement Stok (Inti Otomatisasi Sirkulasi) // Fungsi ini akan menjalankan
```

```
query ke latar belakang: UPDATE buku SET stok = stok - 1 WHERE id = x $buku->decrement('stok'); return redirect()->route('peminjaman.index')->with('success', 'Transaksi berhasil disimpan dan stok dipotong.');
```

Sebaliknya, pada saat buku dikembalikan ke perpustakaan, stok yang tadinya dikurangkan harus dikembalikan. Algoritma ini diletakkan pada metode kustom bernama `kembalikan()`. Proses `*increment*` dapat dieksekusi dengan sangat mulus karena relasi `*buku*` telah dideklarasikan di Model sebelumnya (dapat dipanggil dengan syntax `$peminjaman->buku->increment(...)`).

```
// File: app/Http/Controllers/PeminjamanController.php // LOGIKA SAAT BUKU DIKEMBALIKAN (RESTOCK INVENTARIS) public function kembalikan(Peminjaman $peminjaman) { // Verifikasi Anti-Cheat: Mencegah buku yang statusnya sudah 'Dikembalikan' ditambah stoknya berkali-kali if ($peminjaman->status === 'Dipinjam') { // Memodifikasi nilai rekaman status menjadi Selesai (Dikembalikan) $peminjaman->update(['status' => 'Dikembalikan']); // Memanggil tabel Buku secara relasional, dan menambah (+1) stok kembali // Fungsinya setara dengan UPDATE buku SET stok = stok + 1 WHERE id = x $peminjaman->buku->increment('stok'); return redirect()->route('peminjaman.index')->with('success', 'Buku telah diterima dan stok bertambah.');
```

3.6 Sistem Rendering Laporan Eksternal (Ekspor Data ke File Dokumen)

Untuk menunjang proses audit manajemen administrasi, data-data digital di dalam aplikasi wajib dapat dicetak. Laravel menggunakan integrasi modul pihak ketiga (Third Party Plugins), yaitu DOMPDF dan Laravel Excel untuk mengkonstruksi (Render) tabel HTML ke dalam representasi format PDF (Portable Document Format) dan Spreadsheet XLS.

```
// File: app/Http/Controllers/PeminjamanController.php // METODE EKSPOR PDF MENGGUNAKAN DOMPDF public function exportPdf() { // Metode with() digunakan untuk Eager Loading, artinya sistem akan merangkum query ke tabel // anggota dan buku secara instan untuk mempercepat waktu render dibandingkan pemanggilan Lazy Loading. $peminjamans = Peminjaman::with(['anggota', 'buku'])->latest()->get(); // Engine DOMPDF membaca tampilan desain di folder resources/views/peminjaman/pdf.blade.php // dan mengkonversinya (compile) menjadi tata letak PDF sungguhan. $pdf = Pdf::loadView('peminjaman.pdf', compact('peminjamans')); // Perintah Force Download memaksa peramban (browser) milik pengguna untuk mengunduh dokumen. return $pdf->download('Laporan_Sirkulasi_Peminjaman_'.date('Ymd').'.pdf'); } // METODE EKSPOR EXCEL MENGGUNAKAN MAATWEBSITE EXCEL public function exportExcel() { // Memanggil abstraksi Class PeminjamanExport khusus untuk menangani struktur Cell Row/Col pada Microsoft Excel return Excel::download(new \App\Exports\PeminjamanExport, 'Data_Peminjaman_'.date('Ymd').'.xlsx');
```

BAB IV KESIMPULAN

Pengembangan dan integrasi Sistem Informasi Perpustakaan ini telah mengadopsi standar alur kerja sirkulasi digital secara menyeluruh. Penggunaan *Design Pattern* MVC pada Laravel berhasil mendesentralisasikan kode secara efektif, di mana pengaturan antarmuka `*(Views)*` dipisahkan sepenuhnya dari kontrol keamanan alur bisnis `*(Controllers)*`. Otomatisasi

manipulasi stok buku melalui mekanisme deteksi relasional transaksi peminjaman terbukti mampu menghilangkan selisih stok (inventaris) fisik perpustakaan. Selain itu, fitur pendukung pelaporan instan ke dalam bentuk PDF dan Spreadsheet, serta pembatasan kueri antarmuka (*Pagination & Search*), menjadikan sistem ini siap untuk beroperasi dengan lancar, aman, serta memiliki performa *loading* yang responsif meskipun pada beban volume pangkalan data yang besar di masa depan.